

CHAPTER 04

OpenCV 주요 기능

OpenCV 4로 배우는 컴퓨터 비전과 머신 러닝

4장 OpenCV 주요 기능

4.1 카메라와 동영상 파일 다루기

4.2 다양한 그리기 함수

4.3 이벤트 처리

4.4 OpenCV 데이터 파일 입출력

4.5 유용한 OpenCV 기능

4.2 다양한 그리기 함수

그리기 함수

- 그리기 함수는 영상 위에 직선, 도형, 문자열을 그려주는 기능
- 영상 처리나 컴퓨터비전의 처리결과를 영상에 표시할 때 사용함
- 그리기함수의 모든 점들은 2차원 xy 좌표계를 기준으로 표시 -> Point 객체를 이용

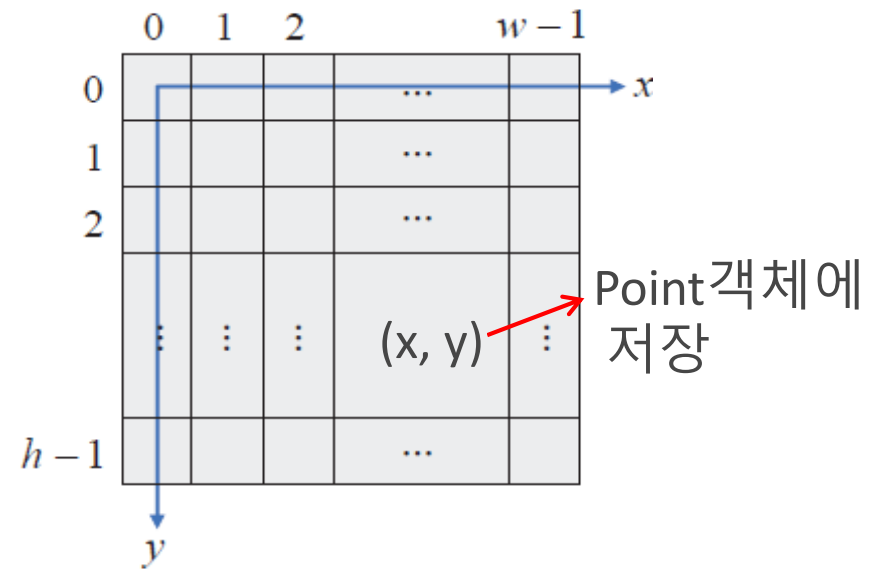
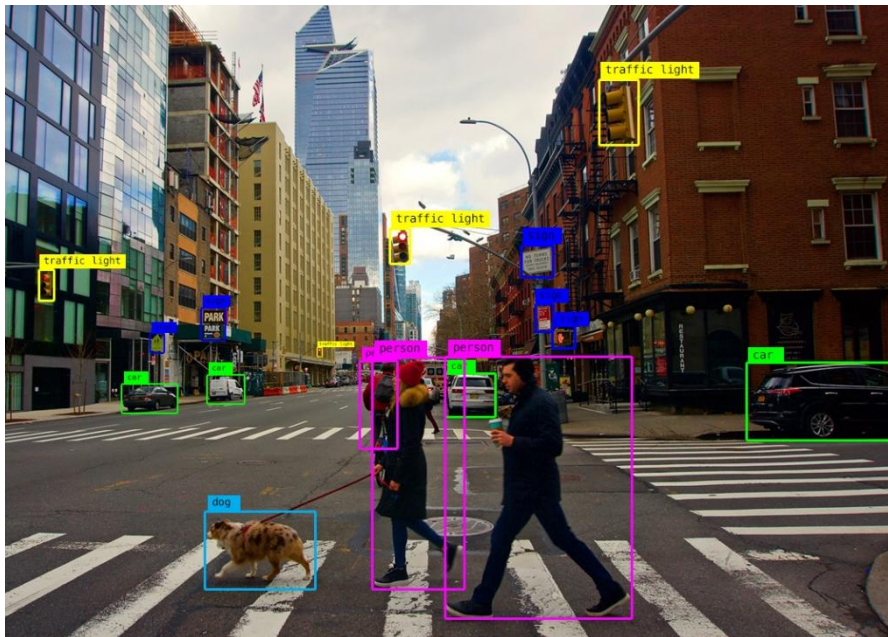


그림 1-4 디지털 영상 표현과 좌표계

직선 그리기

```
void line(InputOutputArray img, Point pt1, Point pt2, const Scalar& color,
          int thickness = 1, int lineType = LINE_8, int shift = 0);
```

- `img` 입출력 영상
- `pt1` 시작점
- `pt2` 끝점
- `color` 선 색상(또는 밝기)
- `thickness` 선 두께
- `lineType` 선 타입. `LINE_4`, `LINE_8`, `LINE_AA` 중 하나를 지정합니다.
- `shift` 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>) 연산)

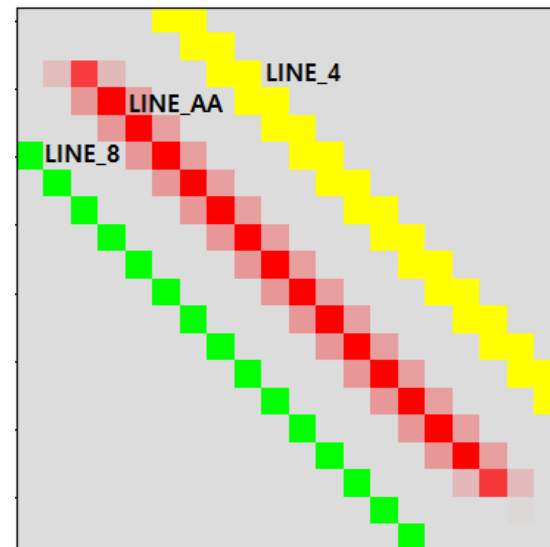
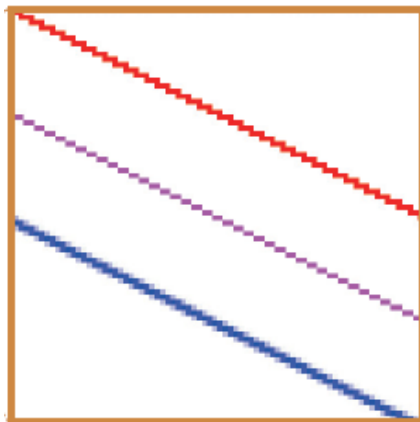
- 영상 위에 직선을 그리는 함수임, 직선에 해당하는 좌표의 `img` 영상의 픽셀값을 수정한다.



직선 그리기

표 4-4 LineTypes 열거형 상수

LineTypes 열거형 상수	값	설명
FILLED	-1	내부를 채움(직선 그리기 함수에는 사용 불가)
LINE_4	4	4방향 연결
LINE_8	8	8방향 연결
LINE_AA	18	안티에일리어싱(anti-aliasing)



직선 그리기

```
void arrowedLine(InputOutputArray img, Point pt1, Point pt2, const Scalar& color,
                 int thickness=1, int line_type=8, int shift=0, double tipLength=0.1);
```

- `img` 입출력 영상
- `pt1` 시작점
- `pt2` 끝점
- `color` 선 색상
- `thickness` 선 두께
- `line_type` 선 타입. `LINE_4`, `LINE_8`, `LINE_AA` 중 하나를 지정합니다.
- `shift` 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>)) 연산)
- `tipLength` 전체 직선 길이에 대한 화살표 길이의 비율

- 화살표 형태의 직선을 그려주는 함수



직선 그리기

```
void drawMarker(InputOutputArray img, Point position, const Scalar& color,
                int markerType = MARKER_CROSS, int markerSize=20, int thickness=1,
                int line_type=8);
```

- `img` 입출력 영상
- `position` 마커 출력 위치
- `color` 선 색상
- `markerType` 마커 종류. `MarkerTypes` 열거형 상수 중 하나를 지정합니다.
- `markerSize` 마커 크기
- `thickness` 선 두께
- `line_type` 선 타입. `LINE_4`, `LINE_8`, `LINE_AA` 중 하나를 지정합니다.

- 다양한 모양의 마커(marker)를 그림, `position` : 마커의 중심의 좌표

+ × ※ ◇ □ △ ▽

직선 그리기

표 4-5 MarkerTypes 열거형 상수

MarkerTypes 열거형 상수	설명
MARKER_CROSS	십자가 모양(+ 모양)
MARKER_TILTED_CROSS	45도 회전된 십자가 모양(x 모양)
MARKER_STAR	MARKER_CROSS 모양과 MARKER_TILTED_CROSS 모양이 합쳐진 형태(* 모양)
MARKER_DIAMOND	마름모 모양(◇ 모양)
MARKER_SQUARE	정사각형 모양(□ 모양)
MARKER_TRIANGLE_UP	위로 뾰족한 삼각형(△ 모양)
MARKER_TRIANGLE_DOWN	아래로 뾰족한 삼각형(▽ 모양)

코드4-6 : 직선그리기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img(400, 400, CV_8UC3, Scalar(255, 255, 255));
    line(img, Point(50, 50), Point(200, 50), Scalar(0, 0, 255));
    line(img, Point(50, 100), Point(200, 100), Scalar(255, 0, 255), 3);
    line(img, Point(50, 150), Point(200, 150), Scalar(255, 0, 0), 10);

    line(img, Point(250, 50), Point(350, 100), Scalar(0, 0, 255), 1, LINE_4);
    line(img, Point(250, 70), Point(350, 120), Scalar(255, 0, 255), 1, LINE_8);
    line(img, Point(250, 90), Point(350, 140), Scalar(255, 0, 0), 1, LINE_AA);

    arrowedLine(img, Point(50, 200), Point(150, 200), Scalar(0, 0, 255), 1);
    arrowedLine(img, Point(50, 250), Point(350, 250), Scalar(255, 0, 255), 1);
    arrowedLine(img, Point(50, 300), Point(350, 300), Scalar(255, 0, 0), 1,
        LINE_8, 0, 0.05);
}
```

코드4-6 : 직선그리기

```
drawMarker(img, Point(50, 350), Scalar(0, 0, 255), MARKER_CROSS);
drawMarker(img, Point(100, 350), Scalar(0, 0, 255),
            MARKER_TILTED_CROSS);
drawMarker(img, Point(150, 350), Scalar(0, 0, 255), MARKER_STAR);
drawMarker(img, Point(200, 350), Scalar(0, 0, 255), MARKER_DIAMOND);
drawMarker(img, Point(250, 350), Scalar(0, 0, 255), MARKER_SQUARE);
drawMarker(img, Point(300, 350), Scalar(0, 0, 255),
            MARKER_TRIANGLE_UP);
drawMarker(img, Point(350, 350), Scalar(0, 0, 255),
            MARKER_TRIANGLE_DOWN);

imshow("img", img);
waitKey();
return 0;
}
```

코드4-6 : 직선그리기

- 점의 좌표를 저장하는 Point 임시객체 생성 : `Point(50, 50)`
- 색상을 저장하는 Scalar 임시객체 생성 : `Scalar(0, 0, 255)`

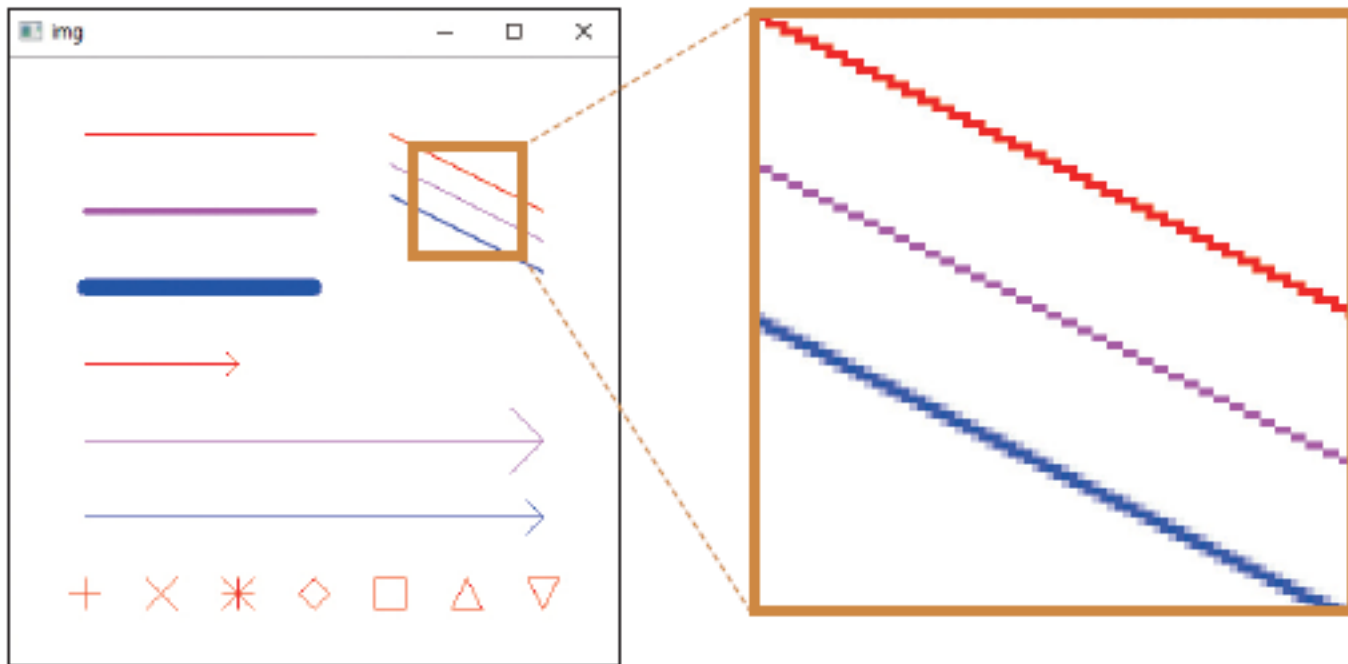


그림 4-4 직선 그리기 예제 실행 결과

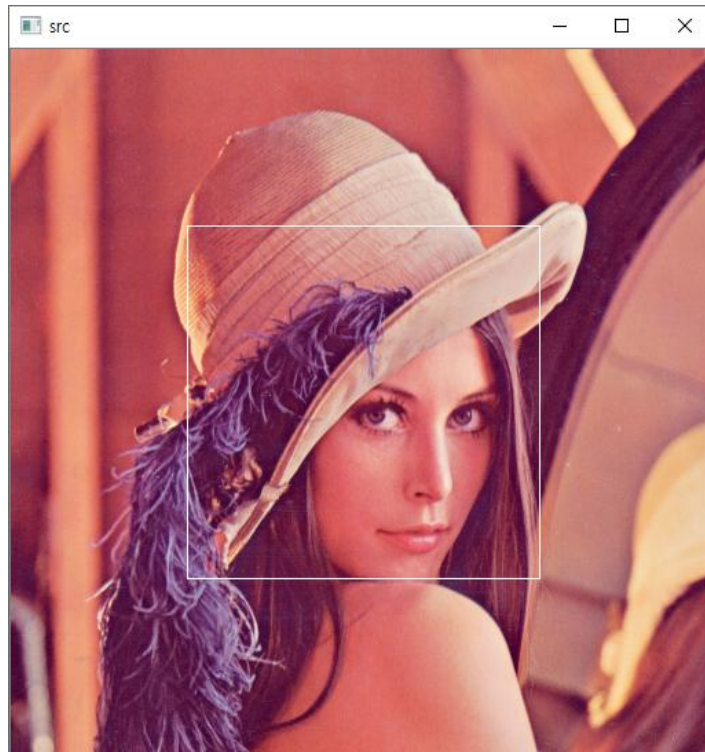
예제: 영상에 직선그리기

```
...
int main(void)
{
    Mat src = imread("lenna.bmp", IMREAD_COLOR);
    if (src.empty()) { cerr << "Image load failed!" << endl; return -1; }

    line(src, Point(src.cols/4, src.rows/4), Point(3*src.cols / 4, src.rows / 4),
          Scalar(255, 255, 255));
    line(src, Point(src.cols/4, 3*src.rows / 4), Point( 3*src.cols / 4, 3*src.rows / 4),
          Scalar(255, 255, 255));
    line(src, Point(src.cols/4, src.rows/4), Point(src.cols / 4, 3*src.rows / 4),
          Scalar(255, 255, 255));
    line(src, Point(3*src.cols/4, src.rows/4), Point(3 * src.cols / 4, 3*src.rows / 4),
          Scalar(255, 255, 255));

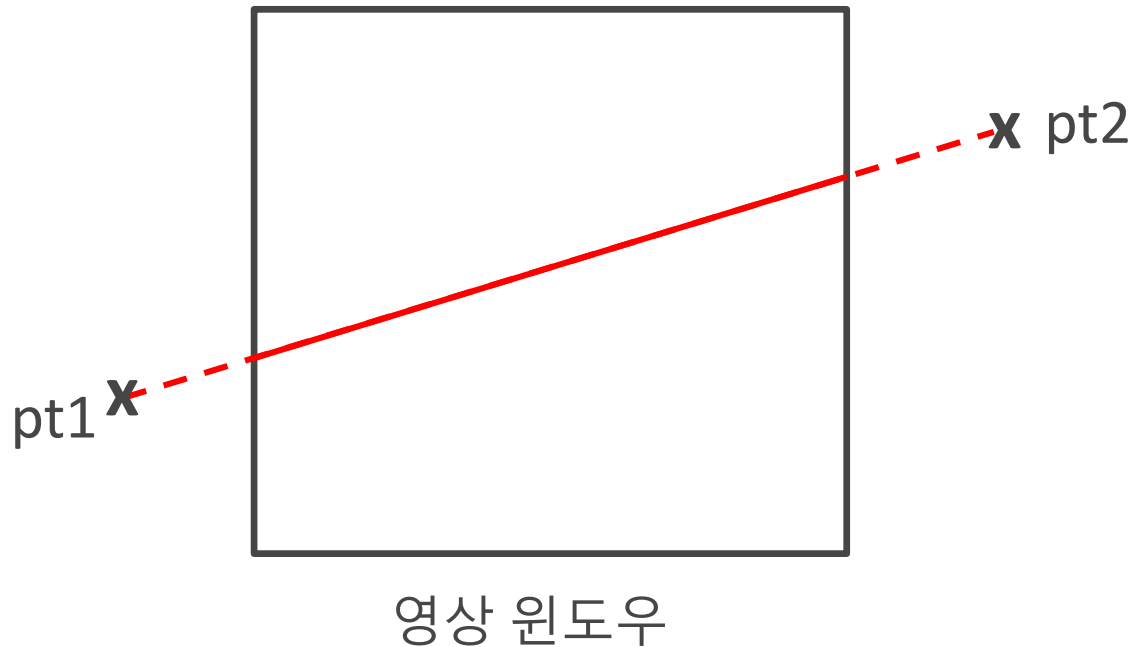
    imshow("src", src);
    waitKey();
    return 0;
}
```

예제: 영상에 직선그리기



주의사항

- 점의 좌표가 영상의 윈도우의 범위를 벗어난 경우 윈도우 밖에도 가상의 좌표계가 있다고 가정하고 그려짐(윈도우안에만 그려짐)



도형 그리기

```
void rectangle(InputOutputArray img, Point pt1, Point pt2, const Scalar& color,
               int thickness = 1, int lineType = LINE_8, int shift = 0);
void rectangle(InputOutputArray img, Rect rec, const Scalar& color,
               int thickness = 1, int lineType = LINE_8, int shift = 0);
```

- **img** 입출력 영상
- **pt1** 사각형 꼭지점 좌표, Point 객체
- **pt2** pt1과 대각 방향에 있는 사각형 꼭지점 좌표, Point 객체
- **rec** 사각형 위치 정보, Rect 객체
- **color** 사각형 색상(또는 밝기)
- **thickness** 사각형 외곽선 두께. 이 값이 음수(-1 또는 FILLED)이면 내부를 채웁니다.
- **lineType** 선 타입
- **shift** 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>) 연산)



pt1, pt2는 서로 대각선 방향에 있어야함

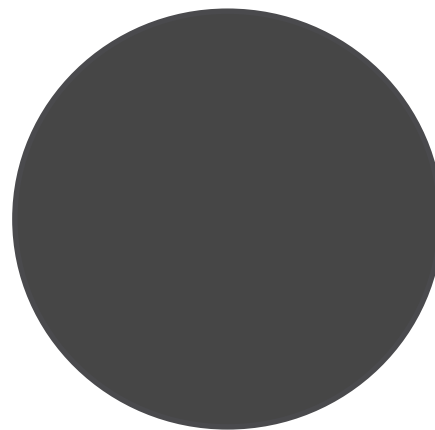
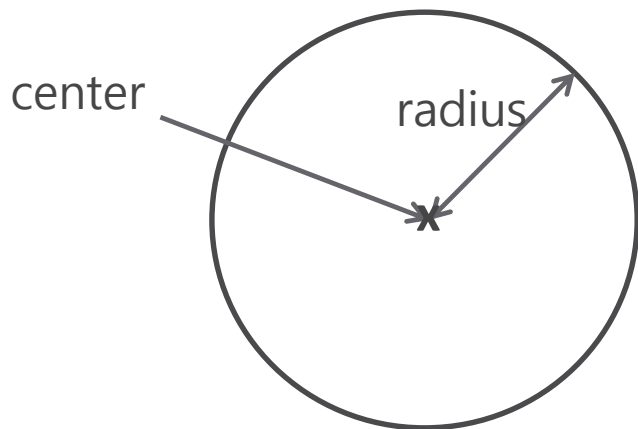


thickness = -1 => 내부를 채움

도형 그리기

```
void circle(InputOutputArray img, Point center, int radius,
            const Scalar& color, int thickness = 1,
            int lineType = LINE_8, int shift = 0);
```

- `img` 입출력 영상
- `center` 원의 중심
- `radius` 원의 반지름
- `color` 원 색상
- `thickness` 원 외곽선 두께. 이 값이 음수(-1 또는 FILLED)이면 내부를 채웁니다.
- `lineType` 선 타입
- `shift` 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>)) 연산



`thickness = -1`
=> 내부를 채움

도형 그리기

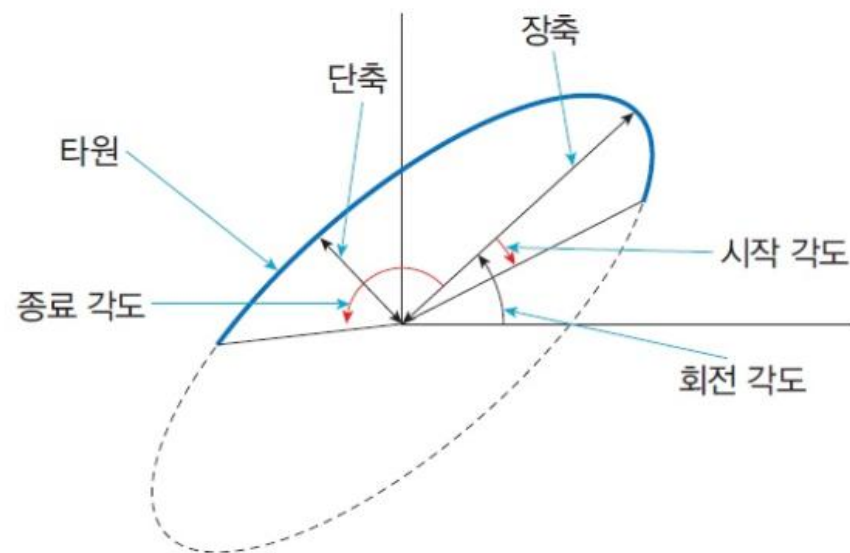
```
void ellipse(InputOutputArray img, Point center, Size axes,
            double angle, double startAngle, double endAngle,
            const Scalar& color, int thickness = 1,
            int lineType = LINE_8, int shift = 0);
```

- **img** 입출력 영상
- **center** 타원의 중심
- **axes** 타원의 반지름. Size(x축_반지름, y축_반지름)
- **angle** 타원 회전 각도(x축 기준, 시계 방향)
- **startAngle** 타원 호의 시작 각도(x축 기준, 시계 방향)
- **endAngle** 타원 호의 끝 각도(x축 기준, 시계 방향)
- **color** 타원 색상
- **thickness** 타원 외곽선 두께. 이 값이 음수(-1 또는 FILLED)이면 내부를 채웁니다.
- **lineType** 선 타입
- **shift** 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>)) 연산

- OpenCV에서 타원을 그리고 싶을 때에는 ellipse() 함수를 사용할 수 있음

도형 그리기

매개 변수	설명
img	타원이 그려질 영상
center	타원의 중심
axes	타원의 장축과 단축
angle	타원의 회전 각도
startAngle	타원의 시작 각도
endAngle	타원의 종료 각도
color	타원 색상
thickness	타원 선의 두께
lineType	타원 선의 타입



도형 그리기

```
void polylines(InputOutputArray img, InputArrayOfArrays pts,
               bool isClosed, const Scalar& color,
               int thickness = 1, int lineType = LINE_8, int shift = 0);
```

- **img** 입출력 영상
- **pts** 다각형 외곽 점들의 좌표 배열. 주로 `vector<Point>` 타입
- **isClosed** 다각형이 닫혀 있는지를 나타내는 플래그. 이 값이 `true`이면 다각형의 마지막 꼭지점과 첫 번째 꼭지점을 잇는 직선을 추가로 그립니다.
- **color** 선 색상
- **thickness** 선 두께. 이 값이 음수(-1 또는 `FILLED`)이면 내부를 채웁니다.
- **lineType** 선 타입
- **shift** 그리기 좌표 값의 축소 비율(오른쪽 비트 시프트(>>)) 연산

- 임의의 다각형을 그리기 위해 `polylines()` 함수를 사용할 수 있음
- `polylines()` 함수에는 다각형의 꼭지점 좌표를 전달해야 하며, 꼭지점 좌표는 보통 `vector<Point>` 자료형에 저장하여 전달함

코드4-7 : 도형 그리기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img(400, 400, CV_8UC3, Scalar(255, 255, 255));

    rectangle(img, Rect(50, 50, 100, 50), Scalar(0, 0, 255), 2);
    rectangle(img, Rect(50, 150, 100, 50), Scalar(0, 0, 128), -1);

    circle(img, Point(300, 120), 30, Scalar(255, 255, 0), -1, LINE_AA);
    circle(img, Point(300, 120), 60, Scalar(255, 0, 0), 3, LINE_AA);

    ellipse(img, Point(120, 300), Size(60, 30), 20, 0, 270, Scalar(255, 255, 0),
            FILLED, LINE_AA);
    ellipse(img, Point(120, 300), Size(100, 50), 20, 0, 360, Scalar(0, 255, 0), 2,
            LINE_AA);
}
```

코드4-7 : 도형 그리기

```
vector<Point> pts;  
pts.push_back(Point(250, 250));  
pts.push_back(Point(300, 250));  
pts.push_back(Point(300, 300));  
pts.push_back(Point(350, 300));  
pts.push_back(Point(350, 350));  
pts.push_back(Point(250, 350));  
polylines(img, pts, true, Scalar(255, 0, 255), 2);  
  
imshow("img", img);  
waitKey();  
return 0;  
}
```

코드4-7 : 도형 그리기

- 원과 타원을 그릴때 선타입을 LINE_AA로 지정하여 그리는 것이 좋음
- 내부를 채울 때는 thickness를 -1로 하거나 lineType을 FILLED 로 설정

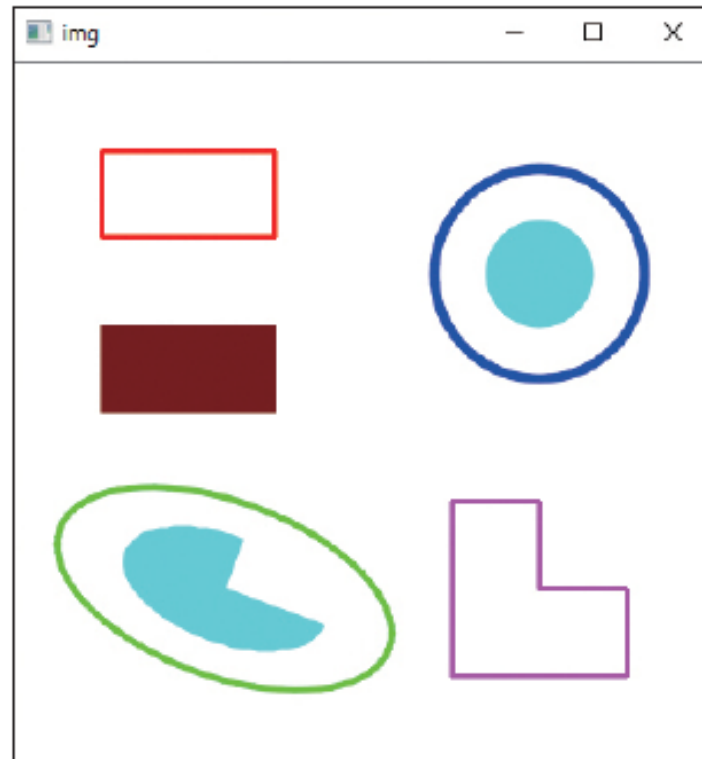


그림 4-5 다양한 도형 그리기 예제 실행 결과

문자열 출력하기

- C++ 표준 입출력을 이용하여 콘솔 창에 문자열을 출력할 수 있음
- 영상에 직접 영상 처리 결과 또는 추가적인 정보를 문자열 형태로 나타내는 것이 보기에 좋음
- OpenCV는 영상 위에 정해진 폰트로 문자열을 출력하는 `putText()` 함수를 제공함
- `putText()` 함수는 입력 영상에서 문자를 출력할 위치의 픽셀값을 원하는 색상으로 수정한다.

문자열 출력하기

```
void putText(InputOutputArray img, const String& text, Point org,
             int fontFace, double fontScale, Scalar color,
             int thickness = 1, int lineType = LINE_8,
             bool bottomLeftOrigin = false);
```

- `img` 입출력 영상
- `text` 출력할 문자열
- `org` 영상에서 문자열을 출력할 위치의 좌측 하단 좌표
- `fontFace` 폰트 종류. `cv::HersheyFonts`에서 선택
- `fontScale` 폰트 크기 확대/축소 비율
- `color` 문자열 색상
- `thickness` 문자열을 그릴 때 사용할 선 두께
- `lineType` 선 타입. `LINE_4`, `LINE_8`, `LINE_AA` 중 하나를 지정합니다.
- `bottomLeftOrigin` 이 값이 `true`이면 영상의 좌측 하단을 원점으로 간주합니다. `false`이면 좌측 상단이 원점입니다.

문자열 출력하기

표 4-6 HersheyFonts 열거형 상수

HersheyFonts 열거형 상수	설명
FONT_HERSHEY_SIMPLEX	일반 크기의 산세리프 폰트
FONT_HERSHEY_PLAIN	작은 크기의 산세리프 폰트
FONT_HERSHEY_DUPLEX	일반 크기의 산세리프 폰트(FONT_HERSHEY_SIMPLEX보다 복잡한 형태)
FONT_HERSHEY_COMPLEX	일반 크기의 세리프 폰트
FONT_HERSHEY_TRIPLEX	일반 크기의 세리프 폰트(FONT_HERSHEY_COMPLEX보다 복잡한 형태)
FONT_HERSHEY_COMPLEX_SMALL	FONT_HERSHEY_COMPLEX보다 작은 폰트
FONT_HERSHEY_SCRIPT_SIMPLEX	필기체 스타일의 폰트
FONT_HERSHEY_SCRIPT_COMPLEX	필기체 스타일의 폰트(FONT_HERSHEY_SCRIPT_SIMPLEX보다 복잡한 형태)
FONT_ITALIC	이탤릭체를 위한 플래그

문자열 출력하기

- 마지막 FONT_ITALIC 상수는 논리합 연산자(|)를 이용하여 다른 HersheyFonts 상수와 함께 사용하며, 이 경우 해당 폰트가 기울어진 이탤릭체로 출력됨 -> FONT_HERSHEY_COMPLEX | FONT_ITALIC
- putText() 함수는 영문자와 숫자를 출력할 수 있으며, 아쉽게도 한글은 출력할 수 없음

코드4-8 : 문자열 출력하기

```
#include <opencv2/opencv.hpp>
#include <iostream>
using namespace cv;
using namespace std;
int main()
{
    Mat img(500, 800, CV_8UC3, Scalar(255, 255, 255));

    putText(img, "FONT_HERSHEY_SIMPLEX", Point(20, 50), FONT_HERSHEY_SIMPLEX, 1,
              Scalar(0, 0, 255));
    putText(img, "FONT_HERSHEY_PLAIN", Point(20, 100), FONT_HERSHEY_PLAIN, 1,
              Scalar(0, 0, 255));
    putText(img, "FONT_HERSHEY_DUPLEX", Point(20, 150), FONT_HERSHEY_DUPLEX, 1,
              Scalar(0, 0, 255));
    putText(img, "FONT_HERSHEY_COMPLEX", Point(20, 200),
              FONT_HERSHEY_COMPLEX, 1, Scalar(255, 0, 0));
    putText(img, "FONT_HERSHEY_TRIPLEX", Point(20, 250), FONT_HERSHEY_TRIPLEX, 1,
              Scalar(255, 0, 0));
    putText(img, "FONT_HERSHEY_COMPLEX_SMALL", Point(20, 300),
              FONT_HERSHEY_COMPLEX_SMALL, 1, Scalar(255, 0, 0));
}
```

코드4-8 : 문자열 출력하기

```
putText(img, "FONT_HERSHEY_SCRIPT_SIMPLEX", Point(20, 350),  
         FONT_HERSHEY_SCRIPT_SIMPLEX, 1, Scalar(255, 0, 255));  
putText(img, "FONT_HERSHEY_SCRIPT_COMPLEX", Point(20, 400),  
         FONT_HERSHEY_SCRIPT_COMPLEX, 1, Scalar(255, 0, 255));  
putText(img, "FONT_HERSHEY_COMPLEX | FONT_ITALIC", Point(20, 450),  
         FONT_HERSHEY_COMPLEX | FONT_ITALIC, 1, Scalar(255, 0, 0));  
  
imshow("img", img);  
waitKey();  
return 0;  
}
```

코드4-8 : 문자열 출력하기



그림 4-6 다양한 폰트로 문자열 출력하기 실행 결과

문자열 출력하기

```
Size getTextSize(const String& text, int fontFace, double fontScale,  
                 int thickness, int* baseLine);
```

- **text** 출력할 문자열
- **fontFace** 폰트 종류
- **fontScale** 폰트 크기 확대/축소 비율
- **thickness** 문자열을 그릴 때 사용할 선 두께
- **baseLine** (출력) 가장 하단의 텍스트 위치를 기준으로 하는 기준선(baseline)의 y 좌표, 필요 없으면 0 지정
- **반환값** 지정한 문자열 출력 시 차지하는 사각형 크기

- `getTextSize()` 함수는 문자열 출력을 위해 필요한 사각형 영역 크기를 계산해준다.
- 이 함수를 잘 이용하면 문자열이 한쪽으로 치우치지 않고 적당한 위치에 출력되도록 설정할 수 있음

코드4-9 : 영상의 중앙에 문자열 출력하기

```
int main()
{
    Mat img(200, 640, CV_8UC3, Scalar(255, 255, 255));
    const String text = "Hello, OpenCV";
    int fontFace = FONT_HERSHEY_TRIPLEX;
    double fontScale = 2.0;
    int thickness = 1;
    Size sizeText = getTextSize(text, fontFace, fontScale, thickness, 0);
    Size sizeImg = img.size();
    Point org((sizeImg.width - sizeText.width) / 2, (sizeImg.height +
        sizeText.height) / 2);
    putText(img, text, org, fontFace, fontScale, Scalar(255, 0, 0), thickness);
    rectangle(img, org, org + Point(sizeText.width, -sizeText.height), Scalar(0,
        255, 0), 1);

    imshow("img", img);
    waitKey();
    return 0;
}
```


코드4-9 : 영상의 중앙에 문자열 출력하기

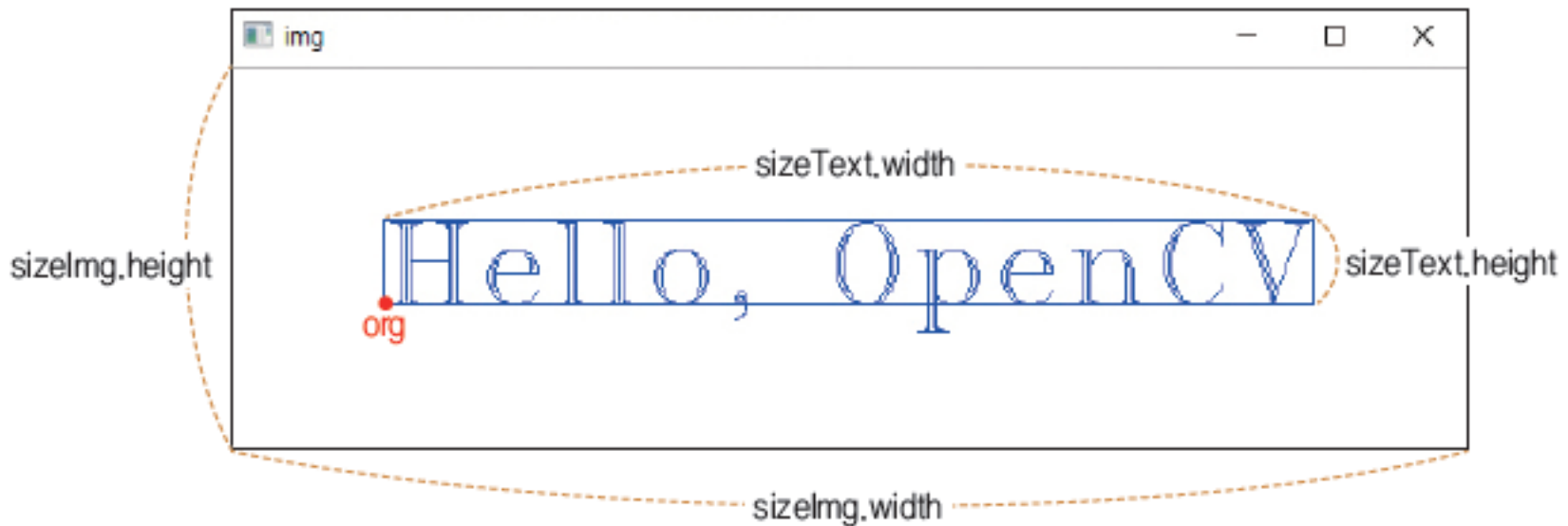
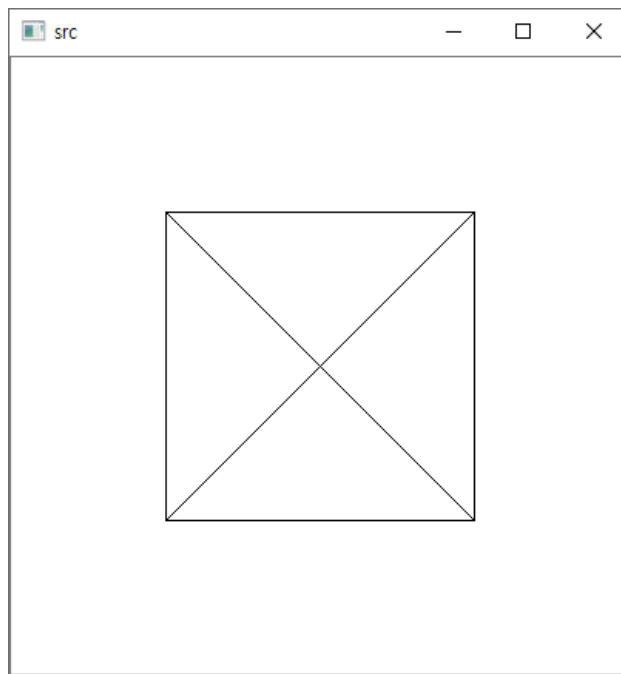


그림 4-7 영상의 중앙에 문자열 출력하기 실행 결과와 동작 원리

실습과제1

- 400x400 영상을 생성하고 다음처럼 line함수를 이용하여 직선을 그리시오.



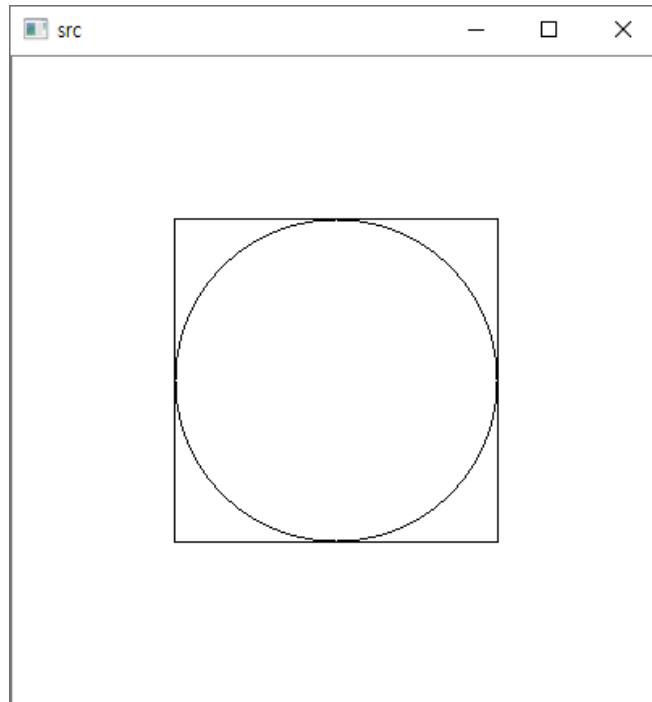
실습과제2

- Lenna 영상에 다음 실행결과처럼 화면을 4등분하는 가로, 세로선을 그리시오. 반드시 line함수를 이용할 것. 라인 스타일 수정 가능함.



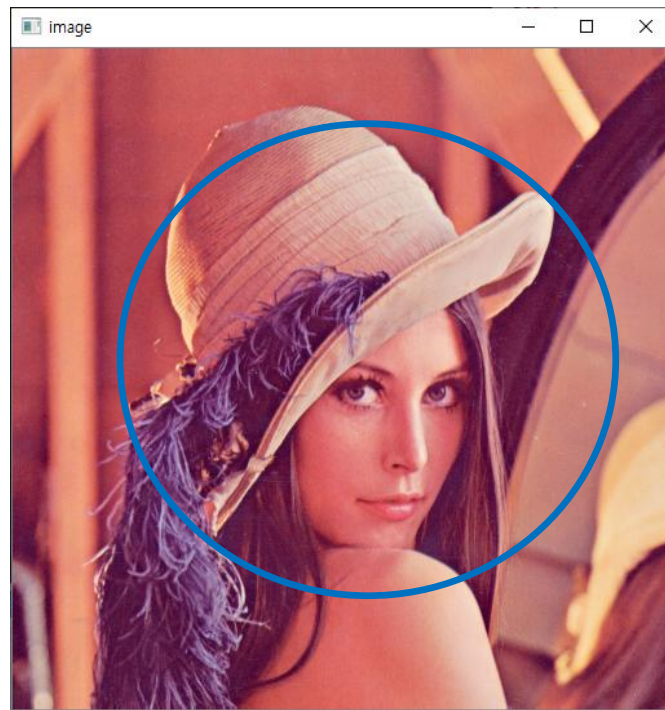
실습과제3

- 400x400 영상을 생성하고 다음처럼 도형을 그리시오.



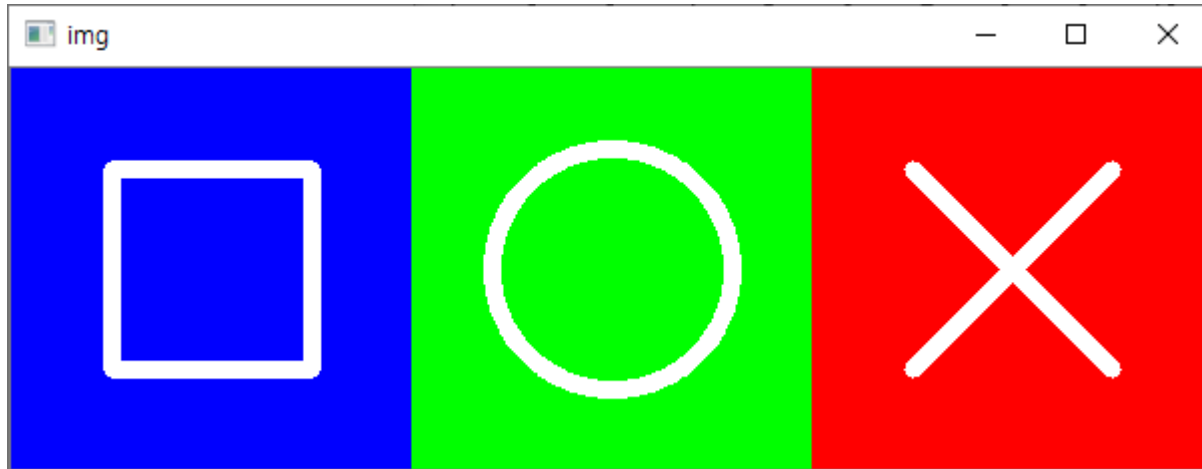
실습과제4

- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 다음 실행결과처럼 얼굴주위에 파란색 원을 그리시오. 필요한 좌표값은 그림판 프로그램을 이용하여 구하시오.



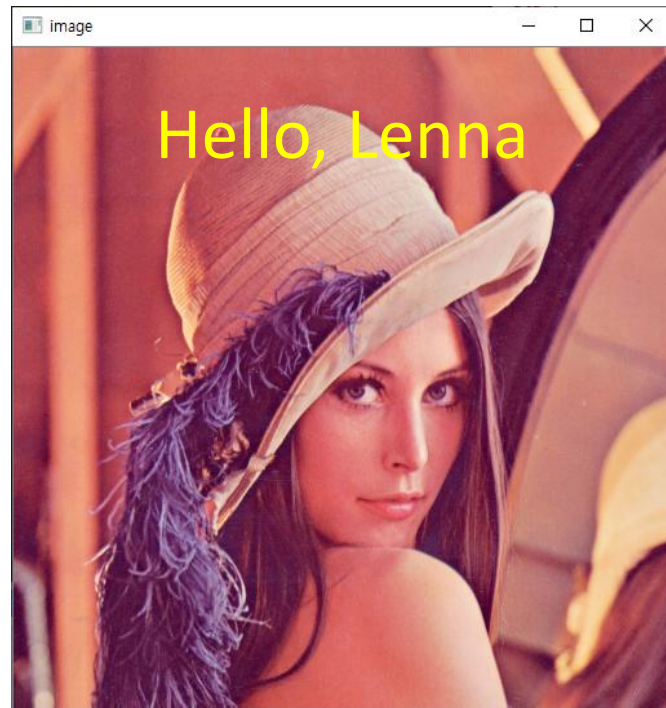
실습과제5

- 600X200 컬러영상을 만들고 다음 실행결과처럼 Red, Green, Blue로 배경을 만들고 각 도형을 그리시오. 힌트: 3장 부분행렬(관심영역)을 응용해볼 것, 3개의 관심영역을 만들어 처리하면 좌표 계산이 쉬움



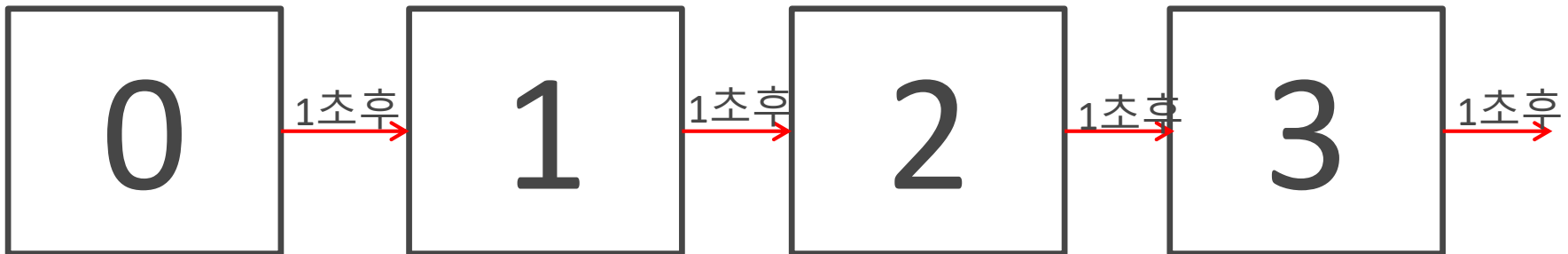
실습과제6

- Lenna 영상을 컬러 영상으로 읽은 후 화면에 출력하고 다음 실행결과처럼 문자열을 출력하시오. 폰트설정은 원하는 대로 하시오.
- 문자열이 상단의 중앙에 위치하도록 위치를 조정하시오.



실습과제7

- 다음처럼 출력하는 초시계를 만들어 보시오.
- 윈도우 사이즈는 300x300으로 만드시오.
- 초기상태는 0을 출력하고 대기한다. 's'키를 누르면 카운트를 시작하고 't'키를 누르면 정지한다. 'r'키를 누르면 0으로 리셋 된다. 'q'키를 입력하면 프로그램을 종료한다.
- 시간제어 및 문자입력을 위하여 waitKey함수를 사용하라.
- 숫자출력을 위하여 변수의 값을 문자열로 변환할 때 format함수를 사용하고 문자열을 출력할 때 putText함수를 사용할 것
- 반복문은 1번만 사용하라. 같은 함수를 2번 이상 호출하지 말 것

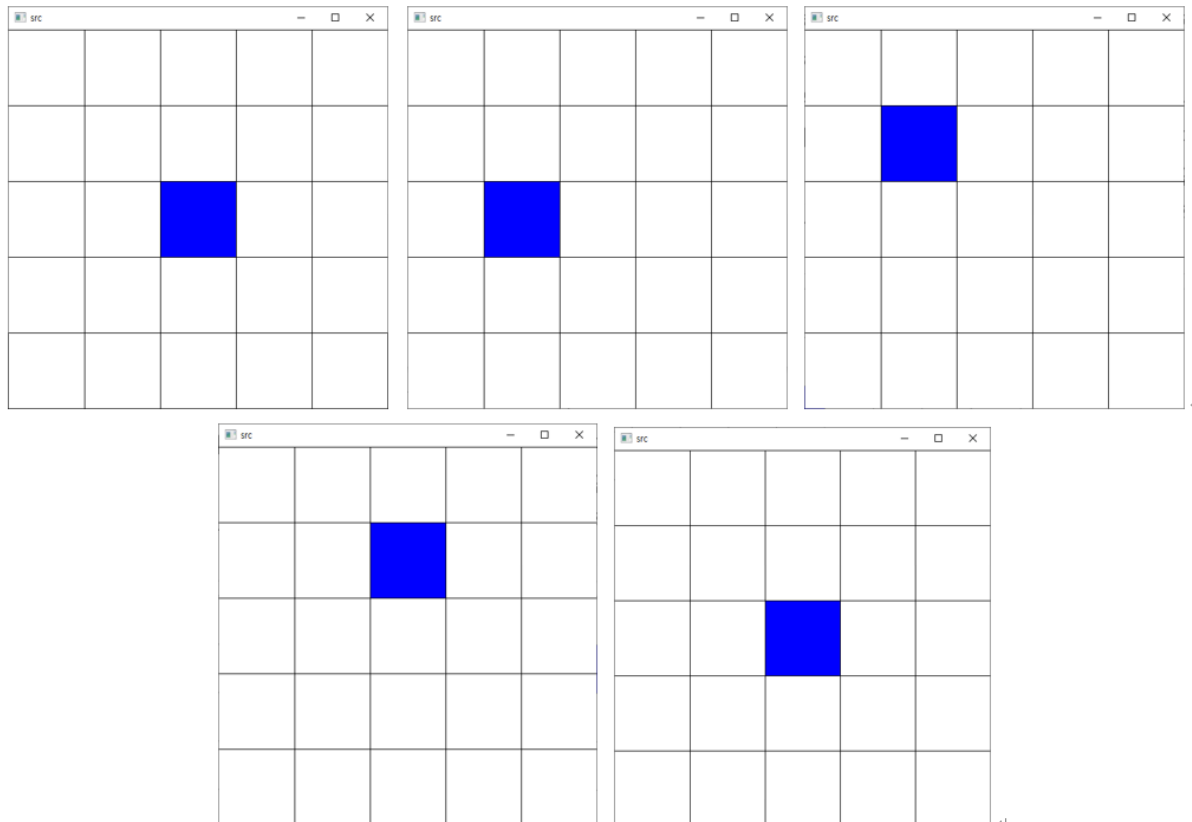


실습과제8

- 키보드를 이용하여 파란색 버튼을 움직이는 프로그램을 만들고자 한다. 아래 지시대로 작성 하시오.
- ① 초기 화면은 폭500, 높이500픽셀의 3채널 컬러영상으로 설정하라. 첫 번째 그림처럼 바둑판모양으로 직선을 출력하고 파란색버튼이 가운데 위치하도록 하라.
- ② 두번째 그림처럼 'l' 키를 누르면 현재위치에서 왼쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ③ 세번째 그림처럼 'u' 키를 누르면 현재위치에서 위쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ④ 네번째 그림처럼 'r' 키를 누르면 현재위치에서 오른쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ⑤ 다섯번째 그림처럼 'd' 키를 누르면 현재위치에서 아래쪽으로 버튼을 한칸 움직인다. 윈도우를 벗어난 경우 현재 위치를 유지하라.
- ⑥ 'q'키를 누르면 종료하도록 하라.

실습과제8

- waitKey함수로 키보드 입력을 받고 새로운 버튼의 좌표를 계산하라. 버튼의 이전 위치도 항상 저장하여 이전 버튼은 지우고 새버튼을 그린다.
- 버튼의 좌표가 윈도우를 벗어난 경우 현재 값을 유지하라.

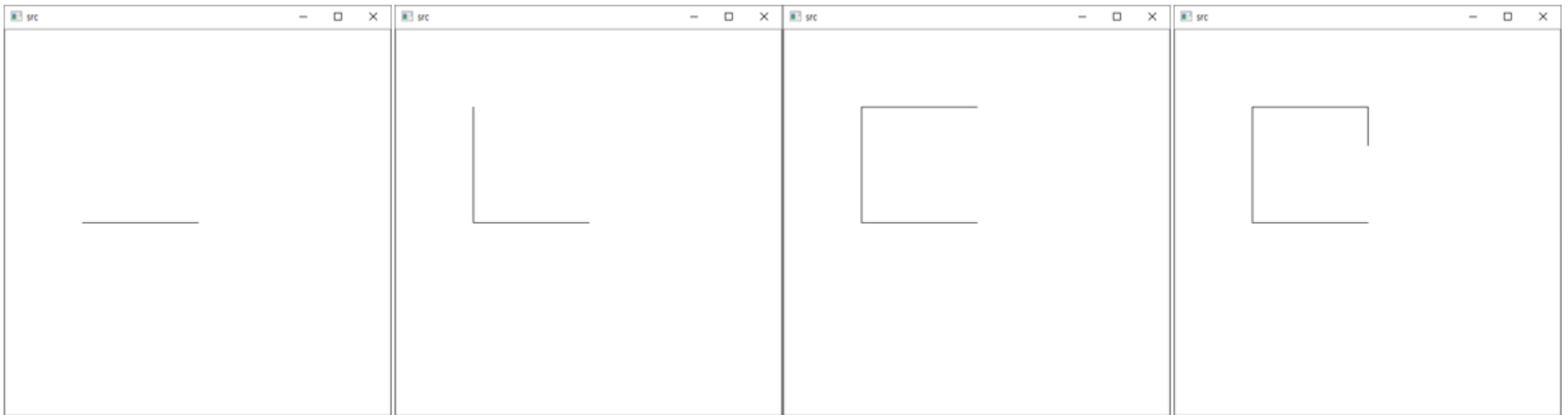


실습과제9

- 키보드를 이용하여 직선을 그리는 프로그램을 만들고자 한다. 아래 지시대로 작성하시오.
- ① 초기 화면은 폭 500, 높이 500픽셀의 3채널 컬러영상으로 설정하라. 직선의 시작 위치는 화면의 중심으로 설정하고 'q'키를 누르면 종료하도록 하라.
- ② 1번 그림처럼 'l' 키를 누르면 현재위치에서 왼쪽으로 길이 50픽셀인 직선을 그려준다.
- ③ 2번 그림처럼 'u' 키를 누르면 현재위치에서 위쪽으로 길이 50픽셀인 직선을 그려준다.
- ④ 3번 그림처럼 'r' 키를 누르면 현재위치에서 오른쪽으로 길이 50픽셀인 직선을 그려준다.
- ⑤ 4번 그림처럼 'd' 키를 누르면 현재위치에서 아래쪽으로 길이 50픽셀인 직선을 그려준다.

실습과제9

- 아래 실행 결과는 l키를 3번 반복, u키를 3번 반복, r키를 3번 반복, d키를 1번 누른 결과이다.



과제 제출 방법

- 소스파일, 라인단위 코드설명, 실행결과를 하나의 PDF 파일로 작성하여 과제게시판에 제출하시오. 이외 양식은 0점 처리함
- 제출기한은 과제 게시판을 참고할 것.